PathScope Interactive Shortest Path Algorithm Visualizer

Design and Analysis of Algorithms Course Project Proposal

Project idea

PathScope is a browser-based educational tool that lets a learner draw a graph, choose an algorithm, and watch every decision made by that algorithm. The same graph can then be tested with other suitable algorithms so that the learner can compare paths, data-structure operations, execution steps, and theoretical time complexity.

Problem being solved

Students often memorize pseudocode and complexity formulas without seeing why an algorithm visits a particular vertex or updates a distance. Existing animations commonly show only one fixed example. PathScope turns any learner-created graph into an interactive experiment and connects each animation step to pseudocode, state changes, and measurable operations.

What the application will do

1. Create directed or undirected, weighted or unweighted graphs by adding, moving, connecting, editing, and deleting nodes.
2. Select a valid algorithm and choose the source and destination vertices where required.
3. Play, pause, step forward, step backward, or change animation speed while the active pseudocode line is highlighted.
4. Display the visited set, frontier, queue or priority queue, distance table, predecessor map, and final path.
5. Run a comparison mode that reports operation counts, path cost, observed running time, and asymptotic complexity.

Algorithms from the syllabus

Algorithm	Use	Main visualization	Complexity
BFS	Shortest path in an unweighted graph	Queue, levels, visited nodes	O(V + E)
DFS	Traversal and reachability baseline	Recursion stack and backtracking	O(V + E)
Dijkstra	Non-negative weighted shortest path	Priority queue and relaxations	O((V + E) log V)
DAG shortest path	Weighted directed acyclic graph	Topological order and relaxations	O(V + E)
Floyd–Warshall	Shortest paths between every pair	Distance-matrix updates	$O(V^3)$

Learning and comparison experience

The interface will explain both the result and the reasoning. A learner can use a preset graph or build a new one, predict the next action, and then reveal the actual step. Invalid choices are explained—for example, Dijkstra is disabled when a negative edge is present, and DAG shortest path first verifies that the graph is acyclic.

Core features

- Side-by-side algorithm playback on the same compatible graph.
- Synchronized highlighting of graph elements, pseudocode, and data structures.
- Complexity panel showing V, E, actual operations, and the expected Big-O bound.
- Random and best-, average-, or worst-pattern graph generators for classroom experiments.
- Challenge mode in which students predict the next visited node or distance update.
- Save, load, import, and export graphs as JSON without requiring an account.

Technology stack

Layer	Technology	Purpose
Frontend	React and TypeScript	Component-based interface with type-safe graph and algorithm models
Build tool	Vite	Fast local development and optimized static web build
Graph canvas	React Flow	Draggable nodes, editable edges, zooming, and custom node states
Styling	Tailwind CSS	Responsive and consistent educational dashboard layout
Charts	Recharts	Comparison charts for operations, time, and graph size
Algorithm core	TypeScript generators	Produces one deterministic state snapshot per logical step
Performance	Web Workers	Runs benchmarks without freezing the animation interface
Storage	LocalStorage and JSON	Saves graphs locally; no server or database needed for the MVP
Testing	Vitest and Playwright	Unit tests for correctness plus browser workflow tests
Deployment	Vercel or GitHub Pages	Simple static deployment with a shareable URL

Why this stack fits the project

The complete application can run in the browser, which keeps implementation and demonstration simple. The algorithm engine remains independent from the interface, so each algorithm can be tested for correctness before its snapshots are animated.

System architecture

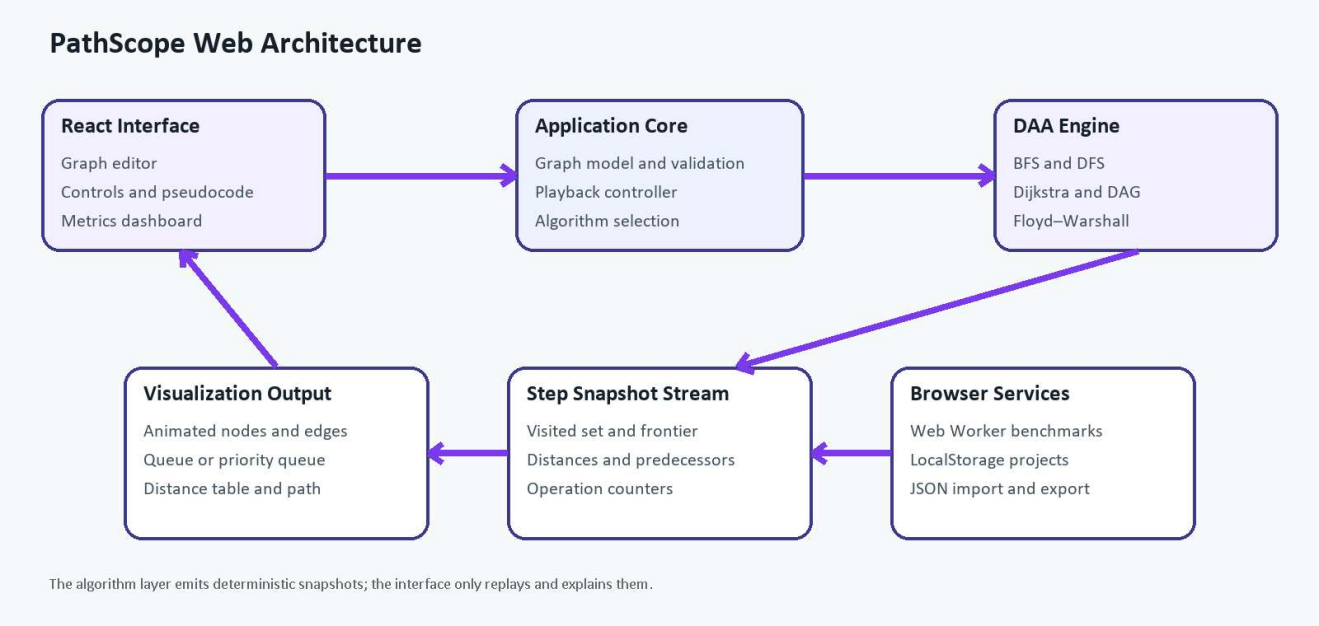


Figure 1 Proposed browser-only architecture

Execution workflow

Stage	Responsibility
1 Input	The user creates a graph or selects a preset and chooses a valid algorithm.
2 Validate	The graph model checks weights, direction, cycles, and required source or destination.
3 Execute	The selected algorithm emits snapshots containing its current state and counters.
4 Replay	The playback controller animates snapshots and synchronizes pseudocode and data structures.
5 Compare	The dashboard summarizes path cost, steps, operations, measured time, and Big-O complexity.

Expected deliverables

- Responsive web application with a graph editor and five syllabus-aligned algorithm modules.
- At least six curated examples covering unweighted, weighted, DAG, dense, and disconnected graphs.
- Comparison dashboard, complexity explanations, challenge mode, and graph import or export.
- Correctness tests that verify paths, distances, traversal order constraints, and invalid-input handling.